

Evaluación del módulo #4

Consigna del trabajo 

Proyecto: Smart Task

Evaluación del módulo 4: Fundamentos de Programación en Java

Situación inicial

Unidad solicitante: Dirección de Tecnología de una empresa de software educativa.

La organización detectó la necesidad de una herramienta sencilla pero funcional que permita a los estudiantes y trabajadores gestionar sus tareas diarias desde la consola, sin necesidad de interfaces gráficas. Se busca una solución que pueda expandirse fácilmente a futuro y que sirva como base para prácticas de buenas prácticas en desarrollo en Java.

Nuestro objetivo

Desarrollar una aplicación de consola en Java llamada SmartTask, que permita gestionar tareas personales (agregar, listar, marcar como completadas y eliminar), aplicando progresivamente los fundamentos del lenguaje Java, algoritmos, estructuras de control, orientación a objetos, principios de diseño, pruebas unitarias y documentación técnica. El proyecto debe estructurarse de forma modular y escalable, aplicando buenas prácticas, permitiendo su posterior evolución hacia interfaces gráficas o web.

Requerimientos

- Uso del lenguaje Java y su entorno de desarrollo (JDK, Eclipse o VSCode).
- Aplicación ejecutable desde consola (.jar).
- Capacidad de:
 - Agregar tareas con nombre y prioridad.
 - Listar tareas activas y completadas.
 - Marcar tareas como completadas.
 - Eliminar tareas por ID.
- Implementación con:
 - Clases e instancias.
 - Encapsulamiento.
 - Herencia y polimorfismo.
 - Interfaces y clases abstractas.
 - Buenas prácticas de codificación y documentación (JavaDoc).

- Pruebas unitarias con JUnit5.

Paso a paso

Este proyecto refiere exclusivamente al **módulo 4: Fundamentos de Programación en Java**, y se compone de **7 etapas (lecciones)**, las cuales podrás avanzar de forma progresiva y escalonada con la ayuda de los manuales teóricos y los contenidos desarrollados en las clases en vivo.

Ten en cuenta de invertir **tiempo asincrónicos** para el desarrollo de cada etapa a modo de poder finalizar el módulo y realizar la entrega formal de tu propuesta. Cualquier consulta que surja compártela en los espacios sincrónicos para resolver las dudas en equipo.

A continuación encontrarás las consignas y tareas a desarrollar:

- **Lección 1: El lenguaje Java y su entorno**

-  **Objetivo:** Analizar las características fundamentales del lenguaje Java para su correcta instalación y uso inicial.
-  **Tareas a desarrollar:**
 - Instalar el JDK 8 o superior.
 - Instalar y configurar el IDE Eclipse o VSCode.
 - Crear el proyecto SmartTask en el IDE.
 - Generar la estructura base con el main() y probar el clásico “Hola Mundo”.

- **Lección 2: Algoritmos**

-  **Objetivo:** Estructurar la lógica de la aplicación SmartTask con algoritmos.
-  **Tareas a desarrollar:**
 - Definir los algoritmos necesarios para agregar, listar y eliminar tareas.
 - Simular los algoritmos en pseudocódigo o diagramas de flujo.
 - Implementar los primeros métodos en Java según dicha lógica.

- **Lección 3: El entorno Java para la programación**

-  **Objetivo:** Utilizar la sintaxis básica del lenguaje Java.
-  **Tareas a desarrollar:**
 - Crear la clase Tarea con atributos como id, nombre, prioridad, completado.
 - Aplicar tipos de datos, operadores y estructuras de control (if, switch, while).
 - Probar funcionalidades básicas en consola.
- **Lección 4:** *Desarrollo de aplicaciones de consola en Java*
 -  **Objetivo:** Implementar una aplicación básica de consola utilizando buenas prácticas.
 -  **Tareas a desarrollar:**
 - Crear menú interactivo en consola con Scanner.
 - Agregar opción para agregar, listar y eliminar tareas.
 - Documentar la clase principal con JavaDoc.
 - Implementar depuración con Eclipse.
- **Lección 5:** *El paradigma de orientación a objetos*
 -  **Objetivo:** Utilizar POO para modelar el comportamiento de la aplicación.
 -  **Tareas a desarrollar:**
 - Aplicar encapsulamiento en la clase Tarea.
 - Crear una clase GestorTareas que contenga la lógica del programa.
 - Incorporar métodos como agregarTarea, listarTareas, marcarComoCompletada.
- **Lección 6:** *Polimorfismo y principios básicos de diseño*
 -  **Objetivo:** Aplicar herencia, polimorfismo e interfaces.
 -  **Tareas a desarrollar:**
 - Crear una interfaz Accionable con métodos comunes para operaciones sobre tareas.

- Implementar clases como TareaNormal y TareaUrgente que hereden de Tarea.
- Usar principios de responsabilidad única y bajo acoplamiento.
- **Lección 7: Pruebas unitarias en Java**
 - 📌 **Objetivo:** Implementar una suite de pruebas unitarias con JUnit5.
 - 📌 **Tareas a desarrollar:**
 - Crear clase de pruebas GestorTareasTest.java.
 - Probar funcionalidades como agregarTarea, listarTareas y marcarComoCompletada.
 - Asegurar una cobertura mínima del 80% del código base.

¿Qué vamos a validar? 🔍

- Correcta aplicación de conceptos de programación orientada a objetos.
- Uso adecuado de estructuras de control y sintaxis Java.
- Estructura modular, mantenible y escalable del código.
- Buenas prácticas: convenciones de nombres, documentación, comentarios y estilo.
- Funcionamiento de todas las funcionalidades requeridas desde consola.
- Pruebas unitarias funcionales y cobertura mínima del 80%.
- Código limpio, documentado, sin errores de compilación o warnings innecesarios.

Referencias 🚧

- Documentación oficial Java
- JUnit5 User Guide
- Java API
- Manuales de cada lección proporcionados en el campus

Recursos 📁

- Videos de clase en vivo
- Blogs técnicos como Baeldung y GeeksForGeeks.
- Canal YouTube Java para Todos.
- Proyecto de ejemplo: [GitHub Java Console Task Manager](#)

Entregables

- Código fuente del proyecto **SmartTask** completo.
- Ejecutable **.jar** funcional desde consola.
- Documento con JavaDoc generado.
- Archivo **.zip** con pruebas JUnit.
- Capturas de pantalla del funcionamiento del programa.
- Informe de cobertura de código (opcional).

Portafolio

Se recomienda incorporar este proyecto en el portafolio personal destacando:

- Uso de buenas prácticas y POO en Java.
- Documentación técnica profesional.
- Aplicación funcional multiplataforma desde consola.
- Ejemplo de pruebas unitarias automatizadas con JUnit.

¡Éxitos!

Nos vemos más adelante

